Texas A&M University

CSCE 642: Reinforcement Learning

Fall 2025

Stock Trading using Reinforcement Learning

Final Project Report

Authors:
Hung Nguyen (737002625)
Rahul Baid (836000171)

Project Resources:
[rl4trading Repository]
[Presentation Video]

January 28, 2026

1 Introduction

Financial markets represent one of the most challenging environments for computational modeling, characterized by extreme stochasticity, non-linearity, and volatility. Unlike static environments often found in classical control theory, stock markets are dynamic and non-stationary systems; the underlying statistical properties of market data shift over time due to macroeconomic cycles, changing investor sentiment, and global events [2]. Consequently, a trading strategy that is profitable in one market regime may fail catastrophically in another. Furthermore, financial time-series data suffers from a notoriously low signal-to-noise ratio, where genuine predictive patterns are often obscured by random market fluctuations [7].

Developing automated systems capable of navigating this complexity to generate consistent profit is a longstanding challenge. This project aims to address this by developing and evaluating a comprehensive suite of deep reinforcement learning (RL) agents for automated stock trading. Utilizing the csce642-deepRL GitHub repository as a foundational codebase, along with the Gymnasium and RLlib libraries, we establish a robust experimental framework to ingest historical market data from providers such as Yahoo Finance and train agents to make autonomous trading decisions.

Our approach treats stock trading as a sequential decision-making problem under uncertainty. We define the agent's state space as a comprehensive vector capturing financial position (balance, shares held) and market conditions, augmented by technical indicators including the Moving Average Convergence Divergence and the Relative Strength Index. Our action space is defined as the set of integers $\{-k, \ldots, k\}$, allowing the agent to determine not just the direction of the trade, but its magnitude. By benchmarking value-based methods [1, 10], policy-gradient methods [9], and actor-critic architectures [3, 8] against a standard buy-and-hold strategy, this project evaluates each model's ability to extract signal from noise and maximize portfolio value.

2 Motivation

The primary motivation for applying reinforcement learning to stock trading stems from the inherent limitations of traditional predictive models when facing the complexity of financial markets. Traditional approaches often rely on supervised learning to forecast explicit future prices. However, due to the stochastic and chaotic nature of market dynamics, explicit price prediction is prone to high error rates and often fails to capture the long-term consequences of an immediate trading decision [7].

Reinforcement learning offers a distinct paradigm shift by focusing on policy optimization rather than prediction. An RL agent does not need to forecast the exact future price; instead, it learns

a policy that maps high-dimensional market states to actions that maximize a cumulative reward signal over time. This allows the agent to internalize complex, non-linear dependencies and adapt to market frictions or regime changes that would break a static supervised model. As a result, this approach allows for the direct optimization of risk-adjusted returns, such as the Sharpe ratio, rather than proxy metrics like Mean Squared Error [1, 7].

Finally, despite the growing body of literature in financial RL and the standardization provided by frameworks like FinRL [5], a specific gap remains in the comparative analysis of discrete action spaces. Existing research frequently focuses on continuous control problems for portfolio management [2] or simplistic binary action spaces [12]. There is a lack of systematic comparison regarding how modern Deep RL algorithms perform when applied to a single-asset, discrete action space that permits variable trade magnitudes. This project bridges that gap, systematically evaluating whether sophisticated agents like Deep Deterministic Policy Gradient [3] and Advantage Actor-Critic [8] can exploit feature-engineered states [12] to outperform Deep Q-learning [6] in a realistic, noisy trading environment.

3 Related Works

3.1 Background

To examine the effectiveness of the application of reinforcement learning to finance, we draw from a body of literature. The Direct Reinforcement framework [7] is a foundational work in financial reinforcement learning that introduced Recurrent Reinforcement Learning. Its key contribution was the use of a reward function based on the differential Sharpe ratio, allowing the model to optimize risk-adjusted returns directly at each time step. To avoid the complexity of a differential Sharpe-based reward, our project differs from this approach by employing a simpler and more interpretable reward signal, which is the change in portfolio value to encourage profitability directly. The Sharpe ratio will instead serve as a final evaluation metric over the entire trading period.

Similarly, portfolio management can extend the Direct Reinforcement framework to a more complex, continuous control setting [2]. Jiang, Xu, and Liang [2] allocate portfolio weights across multiple assets, constrained to sum to one, resulting in a high-dimensional, continuous action space. In contrast, our project focuses on a simpler problem: single-asset or independent multi-asset trading, where the action is a scalar decision, such as buy, sell, and hold. Furthermore, while both our project and this paper use reinforcement learning to maximize financial returns, the paper's reward function is the average logarithmic return, which mathematically models a transaction cost factor, rather than our proposal's simpler reward based on the relative change in portfolio value.

Finally, the novel network architecture, Ensemble of Identical Independent Evaluators [12], serves as a strong methodological baseline for our project, as it validates the approach of using pre-calculated technical indicators, such as Moving Average Convergence/Divergence and Relative Strength Index, as the state space for comparing standard reinforcement learning algorithms. This is done by using a feature-engineered state rather than raw price data. However, our project's action space for buying or selling stocks is more expressive as it controls trade magnitude, while the paper uses a simple binary action space for buying or selling, and our reward function is based on total portfolio value, unlike the paper's immediate log-price difference reward.

3.2 Algorithmic Foundations

Our analysis begins with the value-based methods that form our project's baseline. The Q-learning algorithm [10] is a model-free, off-policy algorithm that seeks to find the optimal action-selection policy by learning an action-value function, implemented by our project. However, while the paper is a theoretical proof that guarantees convergence to the optimal action-value function under the strict assumption of a discrete state-action space and a look-up table representation, our project is an application that tackles a complex, continuous state space. As this algorithm's primary weakness is its native tabular form, which cannot handle our high-dimensional, continuous state space, we will use neural networks as function approximators.

However, Deep Q-learning is known to suffer from overestimation bias, a critical weakness in noisy environments, as it uses the same network to both select and evaluate an action [1]. The Double DQN algorithm addresses this by using the online network to select the best action while using the target network to evaluate it. The primary difference between our projects and this paper's is the application domains, since the paper uses a CNN on pixel-based Atari game states, whereas our proposal applies this established algorithm to a financial trading task using a feature-engineered vector state, which includes MACD and RSI. Given the high noise and stochasticity of financial markets, we believe this stabilization technique will be critical to our agent's performance.

As a counterpoint to value-based methods, our project analyzes policy-gradient algorithms. The REINFORCE algorithm [9] is a key inclusion due to its strength in learning a stochastic policy, which may be advantageous for navigating market randomness. Thanks to the Policy Gradient Theorem, which proves that the gradient of expected reward can be estimated from experience, direct policy optimization with function approximation is possible. The primary difference is that the paper provides a theoretical proof of why these methods converge, whereas our proposal is an applied study that uses this algorithm for the stock trading task. Moreover, our project will directly compare this policy-gradient algorithm family against the value-based methods to compare performance.

As REINFORCE's limitation is its high variance in its gradient estimate, we will implement the Advantage Actor-Critic algorithm as a direct, more advanced evolution [8]. By replacing the high-variance return of basic policy gradients with the advantage function and using a critic network to learn a value function and reduce the variance of the actor's policy gradient, A2C is the most promising agent for developing a stable, stochastic trading policy. Unlike the class lecture, which describes this algorithm as general, domain-agnostic, our project will apply this algorithm to our financial task, using a custom state space with MACD, RSI, and a custom reward function.

One of the main challenges in implementing reinforcement learning in stock trading is matching the algorithm to the action space. To extend deep RL to continuous action spaces, we will implement the Deep Deterministic Policy Gradient algorithm [3], an off-policy, actor-critic method by learning a deterministic actor policy to map states to continuous actions, and a critic that evaluates it, then updates the actor by following the critic's gradient while using Deep Q-learning's replay buffer and target networks for stability. However, the algorithm's strength in sample efficiency is tied to a continuous action space, unlike our project's discrete action space. Hence, our project will discretize DDPG's continuous output.

3.3 Related Frameworks

Our project's architecture is validated by modern frameworks. The FinRL library provides a standard framework for investigating the effectiveness of reinforcement learning in stock trading [5]. Both our project and FinRL frameworks use similar state spaces, which include balance, shares owned, and the opening, high, low, and closing prices. Additionally, our project also uses the change in portfolio value as the primary reward function. Also, while the FinRL framework uses real-world market frictions data, such as transaction costs, market liquidity, and a financial turbulence index for risk management, our project did not consider these factors for the sake of simplicity and to enable a better algorithm comparison, allowing this project to isolate the core performance of the reinforcement learning agents without the confounding variables of market friction.

Moreover, Yang et al. [11] shares our project's foundational components by applying the same family of actor-critic algorithms to a stock trading task with a similar feature-engineered state space and a portfolio-value-change reward function. However, the paper proposes a novel ensemble strategy that combines these algorithms, periodically retraining A2C, DDPG, and PPO, and dynamically selecting the agent with the highest Sharpe ratio for the next trading period, whereas our project aims to compare these algorithms to find the single best one. Furthermore, the paper tackles the stock portfolio by normalizing the action space to a continuous vector, while our proposal defines a discrete action space.

Finally, Lima Paiva et al. [4] offers a compelling architectural blueprint for the future expansion of our research. This paper details a method to integrate sentiment analysis with an actor-critic framework. While this paper is similar in its use of A2C, its core technical difference lies in the state space construction. Our proposal currently uses a state space of technical indicators, whereas this paper introduces SentARL, a model that augments the state by adding an explicit market mood vector. This sentiment vector is technically derived by first training a separate CNN-based extractor on news headlines to get a scalar score, which is then grouped hourly and windowed before being fed to the A2C agent.

4 Methodology

4.1 Codebase

For this project, we used the csce642-deepRL GitHub repository as our starting codebase. We created data pipelines to collect data from providers such as Yahoo! Finance and used the Gymnasium library to develop the environments and compare RL algorithms.

4.2 Training Data

To train the models, we downloaded the historical datasets containing 249 days of stock prices from October 1, 2024 to October 31, 2025 using the Yahoo! Finance API. These datasets contain stock prices for 7 companies, including Apple Inc., Amazon, Google, Meta, Microsoft, NVIDIA, and Tesla. These datasets contain Open, High, Low, Close, Volume daily ticker data. Each dataset is split into 199 days for training and 50 days for evaluating the models.

4.3 State Space

The state space for our reinforcement learning agent is a comprehensive vector designed to capture both the agent's current financial position and the prevailing market conditions at each time step t. This state includes the agent's liquid balance, the quantity of shares owned for each stock, and critical market data such as the opening, high, low, and closing prices. To gauge market activity and liquidity, the trading volume is also incorporated. Finally, to provide deeper insights into market momentum and potential trend reversals, the state also includes key technical indicators, such as the Moving Average Convergence Divergence and the Relative Strength Index.

4.3.1 Initialization Strategy

A significant risk in applying reinforcement learning to fixed historical datasets is overfitting to the trajectory. Since our training data represents a single, static history of market prices, an agent that always initializes at $t = 0$ creates a deterministic path. To force the agent to learn generalizable market dynamics rather than specific dates, the Random Start mechanism in the environment's reset function is implemented. At the beginning of every training episode, instead of resetting the internal clock to $t = 0$, the environment selects a random initial timestamp t_{start} from a uniform distribution:

$$t_{start} \sim \text{Uniform}(0, T - T_{min})$$

where T is the total length of the dataset and T_{min} is the minimum required steps for a valid episode.

We utilized different initialization constraints for each experimental phase:

- Single Stock Trading: We set $T_{min} = T$. This constraint disables the random start for the single-stock case, forcing the agent to train on the full, deterministic daily sequence. This was chosen to test if the agent could master a specific, strong trend.
- Multi-Stock Trading: We set $T_{min} = 50$. This allows the random start mechanism to function fully, selecting starting points anywhere from the beginning of the dataset up to 50 days before the end. This acts as a data augmentation strategy, forcing the agents to learn generalizable hedging policies that work across different market windows, rather than memorizing a specific date sequence.

4.4 Action Space

Our agent's action space, which encompasses all permissible moves within the trading environment, is defined as a discrete set of integers ranging from -k to k. In this framework, the sign of the integer dictates the type of action: a positive value represents buying, a negative value represents selling, and zero corresponds to holding the current position. The absolute value of the integer determines the magnitude of the transaction, where k is the maximum allowable action magnitude. We use an action space $\{-k, ..., -1, 0, 1, ..., k\}$, where k is the maximum number of shares that can be bought or sold in a single step. For instance, an action of +10 would execute a purchase of 10 shares of a stock such as AAPL, whereas an action of -10 would command the agent to sell 10 shares. To simplify the action space, we will assume that action selections for different assets are conditionally independent given a state.

4.5 Transition Function

For our model, the transition probability to the next state S' given state S and action A is deterministic or $P(S_{t+1}|S_t, A_t) = 1$.

4.6 Reward Function

To balance immediate profit with long-term stability, we defined the reward function as the relative change in portfolio value normalized by the old portfolio valuation. The reward r_t at time step t is calculated as:

$$r(s_t, a_t, s_{t+1}) = \frac{v_{t+1} - v_t}{v_t}$$

where v_t is the portfolio value at the current step and v_{t+1} is the value at the next step. This normalization is critical for stabilizing training. Unlike a raw profit reward, which grows in magnitude as the portfolio accumulates wealth, potentially causing exploding gradients, this formulation scales the reward to the current asset base.

4.7 Implemented Algorithms

For our learning algorithm, we evaluated multiple reinforcement learning algorithms to determine the most effective strategy. We used Deep Q-learning (DQN) and Double Deep Q-Learning (DDQN), which excels in complex state spaces. We also implemented the policy-gradient algorithm REINFORCE, a Monte Carlo policy gradient method that directly optimizes the policy weights to maximize expected return. To combine the strengths of both paradigms, we leveraged advanced actor-critic models such as Advantage Actor-Critic (A2C) and the Deep Deterministic Policy Gradient (DDPG). To address the complexity of the financial environment, we tailored the implementation of each algorithm to handle the specific requirements of the action space and the need for robust exploration.

4.7.1 Value-Based: Deep Q-Learning and Double Deep Q-Learning

The Deep Q-Network approximates the optimal action-value function $Q^*(s, a)$. The network architecture consists of a multi-layer perceptron (MLP) that maps the input state vector to a single output vector of size $|A| = 2k + 1$. Each output node represents the Q-value for a specific integer action magnitude (e.g., node 0 corresponds to action $-k$, node k to Hold, and node $2k$ to $+k$).

To facilitate exploration, both Deep Q-Learning and Double Deep Q-Learning utilize an ϵ-greedy strategy. With probability ϵ, the agent selects a random integer from the action space. With probability $1 - \epsilon$, it selects the action corresponding to the maximum Q-value. This is done by sampling

$p \sim \text{Uniform}(0, 1)$ where

$$\pi(s) = \begin{cases} \text{random}(A) & \text{if } p < \epsilon \\ \arg\max_a Q(s, a) & \text{otherwise} \end{cases}$$

For environments involving multiple independent assets, we flattened the space to get a discrete action space.

4.7.2 Policy-Gradient: REINFORCE and Advantage Actor-Critic

Both REINFORCE and Advantage Actor-Critic were implemented to support Multi-Discrete action spaces. Instead of outputting a single scalar, the policy networks output a tensor of shape (Batch, number of assets, number of actions). Both algorithms have a shared backbone where a common multilayer perceptron extracts features from the market state. Finally, both algorithms' actor head projects features to $N \times (2k + 1)$ logits while the critic head for Advantage Actor-Critic estimates the scalar value function $V(s)$.

To handle multi-discrete action spaces, we treated the action selection for each asset as conditionally independent given the state. The network outputs logits for each asset, which are converted into independent categorical distributions. The joint log-probability, assuming independence between assets for computational tractability, required for the gradient update, is calculated as the sum of the individual log-probabilities:

$$\log \pi(\mathbf{a}|s) = \sum_{i=1}^{N} \log \pi(a_i|s)$$

This allows the agent to learn complex portfolio allocations without the exponentially growing output layer required by a single flattened formulation. To prevent overselling or overbuying, we apply strict clipping to limit the maximum buy and sell actions.

To facilitate exploration, we model the joint policy as the product of independent policies for each asset:

$$\pi(\mathbf{a}|s) = \prod_{i=1}^{N} \pi_i(a_i|s)$$

The entropy of a joint distribution of independent variables is the sum of their individual entropies. By summing the entropies, we calculate the total uncertainty of the entire portfolio allocation decision. The entropy calculation is used to minimize the loss function. By adding negative entropy to the total loss:

$$L_{total} = L_{policy} + L_{value} - \beta H(\pi)$$

where β is the entropy coefficient.

4.7.3 Continuous Control: Deep Deterministic Policy Gradient

The Deep Deterministic Policy Gradient algorithm utilizes an Actor-Critic architecture tailored for continuous control. For this algorithm, the actor network maps state s to a continuous value in $[-k, k]$ using a scaled $\tanh$ activation while the critic network maps the pair (s, a) to a Q-value. To stabilize learning, we employed target networks (μ', Q') whose weights are updated via Polyak averaging where $\tau = 0.995$.

Since the Deep Deterministic Policy Gradient algorithm outputs continuous values, but our environment's action space is multi-discrete, the algorithm will output a vector in R^7 as the action. To map this action vector to a multi-discrete action space, we will apply discretization to this vector by rounding each entry in the vector to the nearest integer before clipping to limit to the maximum buy and sell actions.

To facilitate exploration, we add Gaussian Noise to the continuous action output during training:

$$a_t = \mu(s_t|\theta^\mu) + \mathcal{N}(0, \sigma)$$

The magnitude of this noise is controlled by a scaling factor, governing the exploration-exploitation trade-off.

4.8 Experiment Design

To rigorously assess the performance of our reinforcement learning agents, we designed two distinct experimental configurations ranging from a simplified single-agent task to a complex portfolio management problem.

4.8.1 Single Stock Trading

The initial phase of our experiment focuses on a Single-Asset Trading task. We utilized Apple Inc. as the target asset due to its high liquidity and representative market volatility. The primary goal of this experiment is to isolate the agent's ability to learn basic market timing—buying low and selling high—without the confounding variables of portfolio allocation or inter-asset correlations. The state space is reduced to the technical indicators and price data of Apple Inc. only. The action space is a single integer scalar $a \in \{-k, \ldots, k\}$. We expect value-based methods like DQN to perform adequately in this simplified setting, providing a baseline for verifying that our technical indicators provide sufficient signal for learning.

4.8.2 Multi-Stock Trading

The second phase extends the environment to a Multi-Asset Portfolio task. We constructed a unified dataset by joining the historical price data of the seven technology companies: Apple Inc., Microsoft, Google, Amazon, NVIDIA, Meta, and Tesla. This configuration tests the agents' ability to manage a diverse portfolio, identify correlations between tech assets, and hedge risk. The state space expands significantly to include price and indicator data for all 7 assets simultaneously. The action space can be multi-discrete for A2C and REINFORCE or a continuous vector for DDPG, requiring the agent to output a joint action vector $\mathbf{a} = [a_1, a_2, \ldots, a_7]$ at every time step. We expect that policy-gradient methods, which can naturally handle multi-dimensional action spaces via independent output heads, will outperform DQN, which suffers from combinatorial explosion in this setting.

4.9 Evaluation Metric

To objectively compare the agents against the Buy-and-Hold baseline, we utilize three primary metrics.

4.9.1 Cumulative Return Curve

We plot the Total Portfolio Value v_t over time for the entire testing period. This visual metric allows us to inspect the stability of the policy. It reveals whether the agent is generating consistent profits or if the final return is the result of a single lucky high-risk trade. It also helps detect periods of significant loss that a single scalar number might hide.

4.9.2 Final Return Percentage

We calculate the Return on Investment (ROI) to measure the absolute profitability of the strategy over the test horizon:

$$ROI = \frac{v_{final} - v_{initial}}{v_{initial}} \times 100\%$$

where $v_{initial}$ and v_{final} are the portfolio values at the beginning and end of the testing period, respectively. This metric provides a direct comparison of the wealth generated by the RL agent versus the passive Buy-and-Hold strategy.

4.9.3 Sharpe Ratio

To ensure that high returns are not simply the result of reckless gambling, we evaluate the Annualized Sharpe Ratio. This metric adjusts the return by the volatility of the portfolio:

$$S = \frac{R_p - R_f}{\sigma_p}$$

where R_p is the expected portfolio return, R_f is the risk-free rate, and σ_p is the standard deviation of the portfolio's excess return. A higher Sharpe ratio indicates that the agent is generating superior returns per unit of risk taken.

4.10 Hyperparameter tuning

Hyperparameter optimization was conducted using a manual trial-and-error approach to balance model stability with sample efficiency. For each algorithm, we conducted 20 distinct trials. In each trial, we iteratively adjusted critical parameters, such as learning rate, batch size, and discount factor, based on the performance of the previous run.

The primary selection criterion for the final parameters was to identify the model configuration that could most closely approximate or exceed the performance of the Buy-and-Hold baseline. This iterative process prioritized consistency over raw maximum returns, allowing us to filter out configurations that led to exploding gradients or failed to converge within the training budget.

5 Results

We evaluate performance for each model against the standard Buy-and-Hold baseline across two distinct environments, which are single-stock trading and multi-stock portfolio management. The final hyperparameters selected for the Single Stock and Multi-Stock environments are shown in Table 1 and Table 2. The disparity in training duration was chosen to address the trade-off between overfitting and underfitting. In the Single Stock setting, the state space is low-dimensional, so training for too many iterations on a single year of historical data risks memorizing the specific price trajectory and noise artifacts rather than learning generalizable market dynamics.

5.1 Single Stock Trading

As shown in Table 3, the passive Buy-and-Hold strategy achieved the highest performance, with a final return of 19.92% and a Sharpe Ratio of 4.03. Among the reinforcement learning agents, DQN was the top performer, achieving a final portfolio value below $120,000 and 18.35% return,

Parameter	DQN	DDQN	REINFORCE	A2C	DDPG
Episodes	1000	1000	1000	1000	1000
Action Limit (k)	200	200	200	200	200
Initial Cash (i)	\$100,000	\$100,000	\$100,000	\$100,000	\$100,000
Network Layers	[128, 128]	[128, 128]	[128, 128]	[128, 128]	[128, 128]
Learning Rate (α)	1×10^{-3}	1×10^{-4}	1×10^{-3}	5×10^{-4}	1×10^{-4}
Discount Factor (γ)	0.99	0.99	0.99	0.99	0.99
Batch Size	64	128	N/A	N/A	128
Replay Buffer Size	50,000	100,000	N/A	N/A	100,000
Exploration	$\epsilon = 0.5$	$\epsilon = 0.5$	Stochastic	Stochastic	Noise Scale=2.0

Table 1: Hyperparameters for Single Stock Environment

Parameter	Discrete DQN	Discrete DDQN	A2C	REINFORCE	DDPG
Episodes	5000	5000	5000	5000	5000
Action Limit (k)	200	200	200	200	200
Initial Cash (i)	\$100,000	\$100,000	\$100,000	\$100,000	\$100,000
Network Layers	[128, 128]	[128, 128]	[64, 64]	[64, 64]	[64, 64]
Learning Rate (α)	1×10^{-3}	1×10^{-3}	5×10^{-4}	1×10^{-3}	1×10^{-4}
Discount Factor (γ)	0.99	0.99	0.90	0.90	0.99
Batch Size	64	64	100	N/A	128
Replay Buffer Size	10,000	10,000	N/A	N/A	100,000
Entropy Coeff. (β)	N/A	N/A	0.01	0.05	N/A
Exploration	$\epsilon = 0.5$	$\epsilon = 0.5$	Stochastic	Stochastic	Noise Scale=50.0

Table 2: Hyperparameters for Multi-Stock Environment

closely tracking the baseline. The remaining models showed similar return percentage, ranging from 16.05% to 16.24%, all significantly lower than Buy-and-Hold and DQN. Specifically, DDQN achieved a final return of 16.19%, REINFORCE reached 16.24% return, and both A2C and DDPG finished with 16.05% return.

By examining Figure 1, the Cumulative Return Curve for single stock trading visually confirms the dominance of the passive Buy-and-Hold strategy. By examining the line graph, the baseline maintains the highest trajectory throughout the entire test, reaching a final portfolio value of around \$120,000. The DQN agent tracks the baseline most aggressively, mirroring its peaks and valleys. However, a noticeable divergence occurs during the September volatility spike. As a result, while the baseline recovers quickly, the DQN agent hesitates, resulting in a persistent performance gap that ended with a final portfolio value below \$120,000. Finally, the A2C, DDPG, and REINFORCE agents form a tight cluster below the DQN line. Their trajectories are smoother but flatter, indicating a more risk-averse policy that avoided large drawdowns but failed to capitalize on the sustained bullish momentum, ending with final portfolio values above \$115,000.

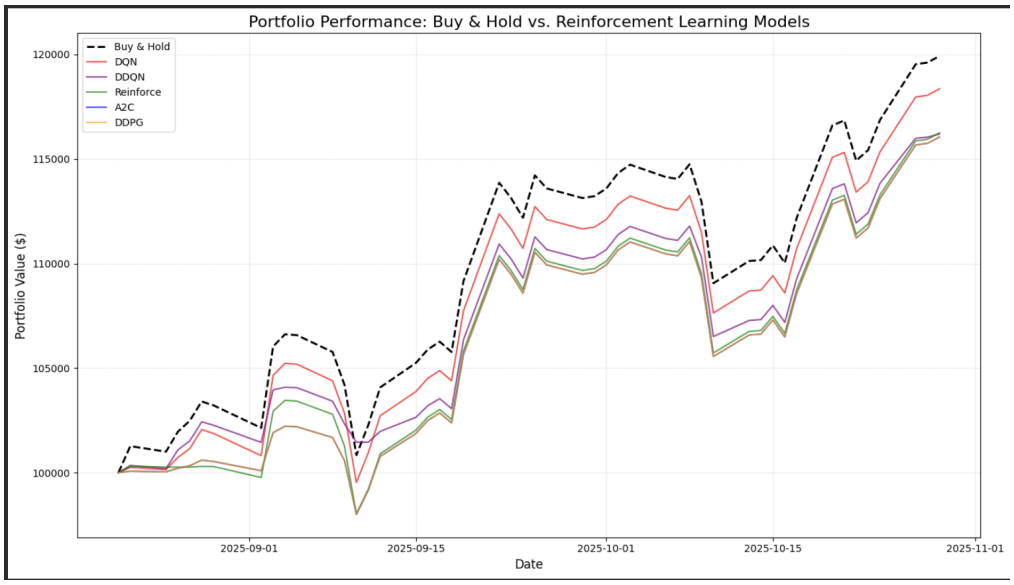

Figure 1: Single Stock Cumulative Return Curve

5.2 Multi-Stock Trading

The Multi-Stock environment, involving the simultaneous management of 7 correlated technology assets, revealed a distinct divergence in algorithmic performance. Unlike the single-stock case, this complex environment highlighted the limitations of value-based methods and the strengths of actor-critic architectures. As observed in Table 3, both DQN and DDQN failed to find a profitable policy, yielding near-zero returns of 0.13% and highly negative Sharpe ratios of -15.63. This failure is vividly illustrated in the Multi-Stock Cumulative Return Curve chart at Figure 2. The DQN and DDQN lines show no noticeable changes and are staying around $100,000. This trend indicates the agents failed to learn any active trading policy, confirming our hypothesis regarding the curse of dimensionality. In the multi-stock trading environment with a flattened action space, the number of possible discrete actions scales exponentially. As a result, the DQN agents could not explore this vast combinatorial space sufficiently to find a profitable policy within the training budget.

In contrast, the remaining reinforcement learning agents, except for the REINFORCE agent, outperformed the Buy-and-Hold baseline, which ended at around $115,000 with a Sharpe ratio of 3.09. The REINFORCE agent demonstrated solid performance and achieved a 14.21% return with a Sharpe Ratio of 2.67. Its trajectory shows it was able to learn the general trend but failed to execute the precise timing required to beat the market.

The A2C agent achieved the highest total profit with a 32.28% return, nearly doubling the Buy-and-Hold return of 15.72%, having a Sharpe ratio of 4.39. This agent's line in Figure 2 tracks the baseline's line closely until around mid-October when a divergence occurs: while the baseline dips, the A2C line surges upward. This decoupling continues through the end of the episode, where the

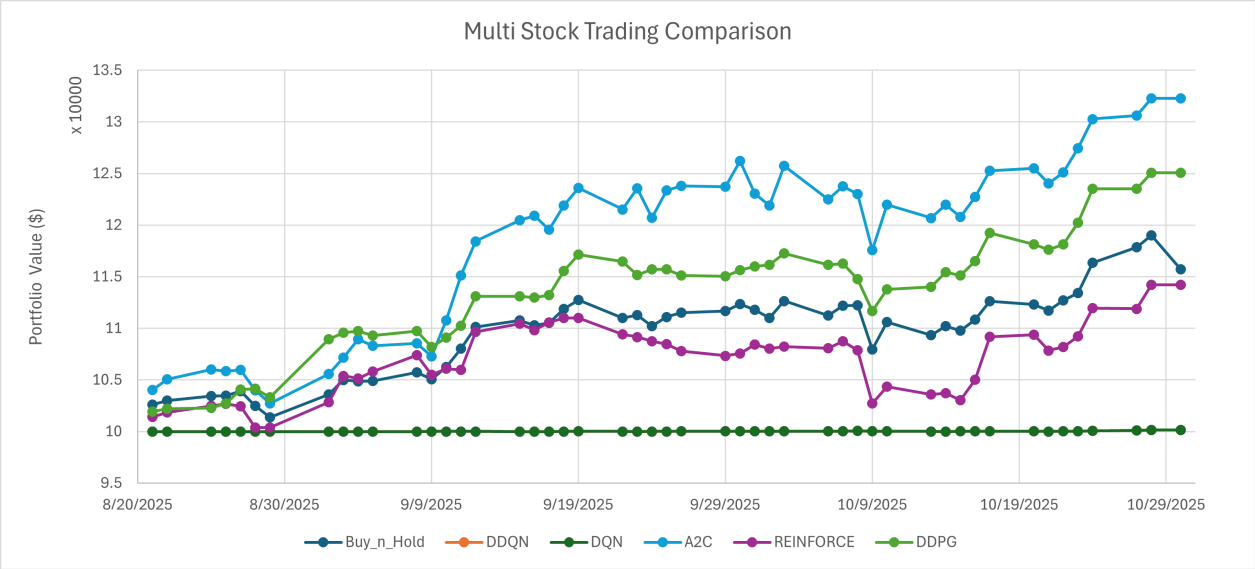

Figure 2: Multi-Stock Cumulative Return Curve

A2C line ends significantly higher than the baseline, suggesting the agent successfully timed the market volatility, ending with a portfolio value above $130,000.

While DDPG's total return of 25.07% was slightly lower than A2C, it achieved a remarkable Sharpe Ratio of 5.11, significantly higher than both A2C's and the baseline's. This agent's line closely follows the upward trajectory of A2C but exhibits less variance. Unlike the A2C line, which has sharp peaks and dips indicating high volatility, the DDPG line is visibly smoother and ended at around $125,000, effectively smoothing out the market noise that affected the Buy-and-Hold strategy.

	Algorithm	*Return on Investment (%)*	*Sharpe Ratio*
Single Stock Trading	A2C	16.05	3.56
	DDPG	16.05	3.58
	REINFORCE	16.24	3.76
	DQN	18.35	3.68
	DDQN	16.19	3.77
	Buy-and-Hold	19.92	4.03
Multi-Stock Trading	A2C	32.28	4.39
	DDPG	25.07	5.11
	REINFORCE	14.21	2.67
	DQN	0.13	−15.63
	DDQN	0.13	−15.63
	Buy-and-Hold	15.72	3.09

Table 3: Overview of Implemented Algorithms

6 Discussion

6.1 Single Stock Trading Analysis

The results from the Single-Stock environment highlight a scenario where the complexity of Deep Reinforcement Learning yielded diminishing returns. The market's strong directional bias (a sustained bull run) favored the passive Buy-and-Hold strategy. The slight underperformance of the RL agents, specifically A2C and DDPG, suggests that in low-dimensional settings, these complex architectures may over-regularize. By attempting to minimize risk, the agents acted too conservatively, exiting positions prematurely and missing the full upside of the rally. As a result, DQN outperformed other reinforcement learning algorithms. However, while the DQN agent achieved a higher total return, its lower Sharpe ratio indicates that these gains were accompanied by significantly higher volatility and larger risk exposure. In contrast, the DDQN agent produced more stable, risk-adjusted returns, demonstrating superior robustness. Furthermore, despite DDPG's design for continuous action spaces, constraining it to a discrete action set resulted in performance comparable to A2C in both return and Sharpe ratio. This suggests that under a simplified, discrete trading regime, algorithmic complexity does not significantly affect outcome, and that performance is instead bounded by the state representation. Finally, across the testing period, the Buy-and-Hold strategy outperformed all RL agents in both absolute return and Sharpe ratio. This suggests that the Apple stock exhibited strong trending behavior, favoring passive investment over active decision-making. The results indicate that, within short and regime-specific datasets, the performance of RL-based trading systems may be constrained not by algorithmic sophistication but by the underlying market structure and informational limits of the state space.

6.2 Multi-Stock Trading Analysis

The Multi-Stock environment validated the core hypothesis of this research: that Policy-Based and Actor-Critic architectures are fundamentally required for high-dimensional portfolio management. The failure of DQN illustrates the curse of dimensionality, where flattening a 7-asset portfolio into a single integer action creates a combinatorial explosion in the action space size. This results in a sparse reward landscape where the probability of randomly selecting a coherent portfolio allocation is near zero. Consequently, the DQN agents failed to learn any active policy, defaulting to the cash-holding behavior observed in Figure 2. The success of A2C proves that the Multi-Discrete formulation is the correct approach. By factoring the action space into independent probability heads, the agent could learn per-asset policies efficiently. However, the significant gap between REINFORCE and A2C highlights the value of the critic network. REINFORCE relies on high-variance Monte Carlo returns,

leading to slower, noisier updates. By introducing a Critic to estimate the value function, A2C reduced this variance, allowing for more aggressive policy updates that outperformed REINFORCE. Finally, DDPG demonstrated the unique advantage of continuous control, which leveraged its architecture to fine-tune its exposure, resulting in the highest risk-adjusted returns, further confirming that minimizing variance, either through a Critic or deterministic continuous control, is the key to mastering volatile financial environments.

6.3 Strengths of Current Approach

Our methodology presents several distinct advantages that contribute to the validity of the results. First, the design of a simplistic, feature-engineered state space effectively mitigates the low signal-to-noise ratio inherent in financial data. By feeding the agents pre-calculated technical indicators like MACD and RSI rather than raw, high-frequency price feeds, we reduced the dimensionality of the learning problem. This allowed the agents to focus on established market signals rather than overfitting to random price fluctuations or noise.

Second, the project utilized a comprehensive evaluation framework that balanced profitability with stability. By not relying solely on raw Cumulative Return, which can incentivize reckless gambling behavior, we incorporated the Sharpe Ratio and visual trajectory inspection. This multi-faceted evaluation ensured that successful agents, such as DDPG, were identified not just by how much money they made, but by their superior risk-adjusted returns and ability to hedge against volatility.

Finally, the experimental design featured a robust testing suite spanning the full spectrum of reinforcement learning paradigms. By implementing and comparing on-policy methods, such as REINFORCE and A2C, with off-policy methods, such as DDPG, DQN, and DDQN, we ensured that our findings were not artifacts of a single algorithmic architecture. This systematic comparison provided strong evidence that Actor-Critic architectures are fundamentally better suited for the high-dimensional, multi-stock trading domain than value-based baselines.

6.4 Limitations of Current Approach

Despite the success of the actor-critic agents, several limitations in our current approach must be acknowledged. A primary constraint is the limited dataset size, which spanned only one year of historical data from October 2024 to October 2025. This restricted time frame exposes the agents to a limited range of market regimes, in which an agent trained on a predominantly bullish year may fail to generalize to a recession or a high-volatility bear market. A more robust agent would require training over a multi-year horizon encompassing diverse economic cycles.

Also, as the hyperparameter tuning process was conducted primarily through manual trial and error, key parameters, such as the learning rate, discount factor, and entropy coefficients, were tuned heuristically rather than through a systematic search method. Consequently, it is likely that the reported performance of agents like DQN or A2C is suboptimal and could be improved with rigorous, automated tuning.

Furthermore, the reward function employed was relatively simplistic, defined as the change in portfolio value normalized by the old portfolio valuation. While interpretable, this signal differs from established financial RL literature, which often suggests optimizing the Differential Sharpe Ratio directly [7] or using logarithmic returns to model continuous compounding [2]. A more sophisticated reward function could allow the agent to optimize for the Sharpe ratio directly within the policy gradient update, potentially leading to even more stable trading behaviors.

6.5 Future Work

Future iterations of this research should focus on integrating unstructured data to enhance the agent's predictive capabilities. The most promising avenue is the integration of Sentiment Analysis using Large Language Models. Stock prices are often driven by news cycles, earnings calls, and global events that are not immediately reflected in technical indicators. As demonstrated by Lima Paiva et al. [4], augmenting the state space with an explicit market sentiment vector derived from financial news headlines can significantly improve agent performance. By employing a pre-trained Large Language Model to generate these sentiment scores, the agent could learn to anticipate market movements based on investor sentiment before they manifest in price action.

7 Conclusion

This project set out to investigate the viability of Deep Reinforcement Learning for automated stock trading, specifically aiming to bridge the gap between theoretical algorithm design and practical application in discrete, k-level action spaces. By developing a comprehensive testing suite that integrated foundational value-based methods with modern actor-critic architectures, we systematically evaluated how different RL paradigms handle the stochasticity and complexity of financial markets.

Our results highlight a clear dichotomy in algorithmic performance governed by environmental complexity. In the simplified Single-Stock domain, we observed that the added complexity of reinforcement learning yielded diminishing returns; the passive Buy-and-Hold strategy outperformed all agents during a strong bull market, suggesting that for single assets with strong upward momentum, attempting to time the market is inferior to passive investment. However, the true strength of Deep

Reinforcement Learning emerged in the complex Multi-Stock environment. In this environment, the curse of dimensionality caused standard value-based methods like DQN and DDQN [1, 10] to fail, while Actor-Critic methods thrived. Most notably, this report validates the effectiveness of two distinct architectural adaptations. First, the Advantage Actor-Critic algorithm [8] is capable of navigating high-dimensional action spaces to maximize raw profit, outperforming the market baseline. Second, the adapted Deep Deterministic Policy Gradient [3], utilizing our custom noise-injection and quantization strategy, demonstrated exceptional stability, achieving risk-adjusted returns that far exceeded the market's performance.

Hence, while reinforcement learning is not effective for every trading scenario, this work demonstrates that Actor-Critic architectures are fundamentally better and more robust than traditional Q-learning approaches for portfolio management. They successfully balance the trade-off between exploration and exploitation, leveraging technical indicators [12] to hedge risk in ways that static strategies cannot.

References

1. Hado van Hasselt, Arthur Guez, and David Silver. “Deep reinforcement learning with double Q-Learning”. In: *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*. AAAI’16. Phoenix, Arizona: AAAI Press, 2016, pp. 2094–2100.
2. Zhengyao Jiang, Dixing Xu, and Jinjun Liang. *A Deep Reinforcement Learning Framework for the Financial Portfolio Management Problem*. 2017. arXiv: 1706.10059 [q-fin.CP]. URL: https://arxiv.org/abs/1706.10059.
3. Timothy P. Lillicrap et al. *Continuous control with deep reinforcement learning*. 2019. arXiv: 1509.02971 [cs.LG]. URL: https://arxiv.org/abs/1509.02971.
4. Francisco Caio Lima Paiva et al. “Intelligent trading systems: a sentiment-aware reinforcement learning approach”. In: *Proceedings of the Second ACM International Conference on AI in Finance*. ICAIF’21. ACM, Nov. 2021, pp. 1–9. DOI: 10.1145/3490354.3494445. URL: http://dx.doi.org/10.1145/3490354.3494445.
5. Xiao-Yang Liu et al. *FinRL: A Deep Reinforcement Learning Library for Automated Stock Trading in Quantitative Finance*. 2022. arXiv: 2011.09607 [q-fin.TR]. URL: https://arxiv.org/abs/2011.09607.
6. Volodymyr Mnih et al. *Playing Atari with Deep Reinforcement Learning*. 2013. arXiv: 1312.5602 [cs.LG]. URL: https://arxiv.org/abs/1312.5602.
7. J. Moody and M. Saffell. “Learning to trade via direct reinforcement”. In: *IEEE Transactions on Neural Networks* 12.4 (2001), 875–889. DOI: 10.1109/72.935097.
8. G. Sharon. *A2C*. Canvas. Texas A&M University. Nov. 2025. URL: https://canvas.tamu.edu/courses/405250/pages/a2c?module_item_id=13300982.
9. Richard S. Sutton et al. “Policy gradient methods for reinforcement learning with function approximation”. In: *Proceedings of the 13th International Conference on Neural Information Processing Systems*. NIPS’99. Denver, CO: MIT Press, 1999, pp. 1057–1063.
10. Christopher J. Watkins and Peter Dayan. “Q-learning”. In: *Machine Learning* 8.3–4 (May 1992), 279–292. DOI: 10.1007/bf00992698.
11. Hongyang Yang et al. *Deep Reinforcement Learning for Automated Stock Trading: An Ensemble Strategy*. 2025. arXiv: 2511.12120 [q-fin.TR]. URL: https://arxiv.org/abs/2511.12120.
12. Alhassan S. Yasin and Prabdeep S. Gill. *Reinforcement Learning Framework for Quantitative Trading*. 2024. arXiv: 2411.07585 [q-fin.TR]. URL: https://arxiv.org/abs/2411.07585.